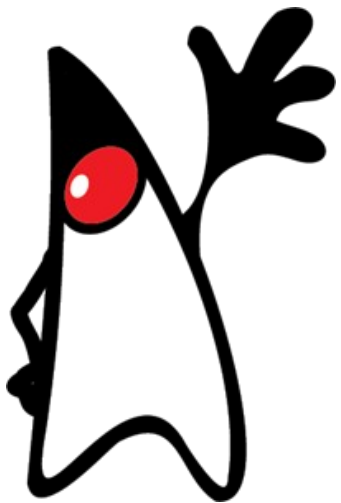




Java API



- Java library: how to use it?
- Class String
- Wrapper classes
- Exercises on use API
- Class Math
- Class LocalDate
- LocalDate and DateTimeFormatter methods applied

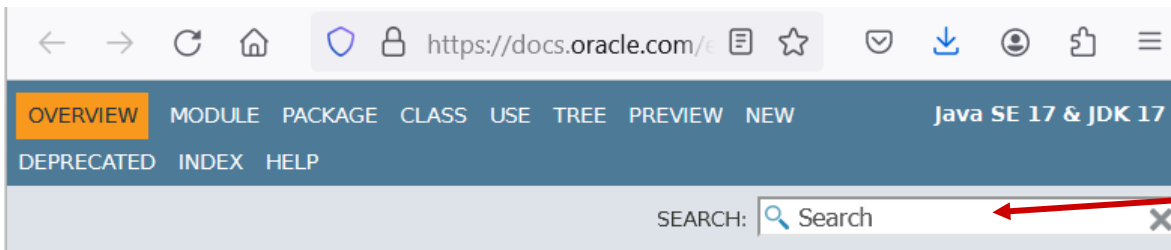
Java API: Application Programming Interface describes the Java library:

- is a collection of prewritten classes, interfaces, and methods
- are organized in packages
 - `java.lang`, `java.util`, `java.io`, `java.math`, `java.net`, `java.sql`, `java.text`, etc.
- written in OO and fully tested
- each class contains a large numbers of methods
- gives the possibility to use functionalities without having to program them yourself (cfr hardware)
- a good Java programmer:
 - knows the most important classes of the library
 - can find the other classes and find information about them

Consult class documentation



- <https://docs.oracle.com/en/java/javase/17/docs/api/index.html>
- Or google: "java api 17 doc"
- Web browser launches the start page of the Java API documentation



To search a class
E.g. String

Java® Platform, Standard Edition & Java Development Kit Version 17 API Specification

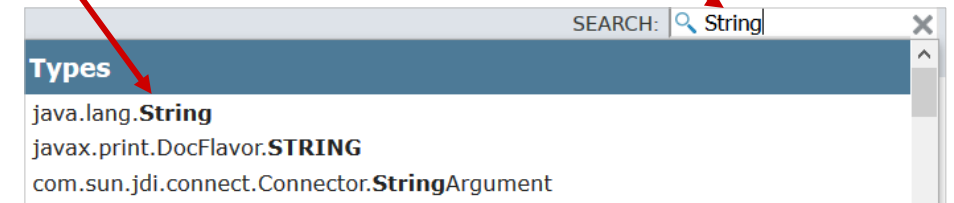
This document is divided into two sections:

Java SE

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with `java`.

JDK

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with `jdk`.



API class String



Attributes

Constructors

Methods

OVERVIEW MODULE PACKAGE **CLASS** USE TREE DEPRECATED INDEX HELP

ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

Module java.base
Package java.lang
Class String

java.lang.Object
 java.lang.String

All Implemented Interfaces:
Serializable, CharSequence, Comparable<String>

Constructors of the class String



Constructor Summary

Constructors

Constructor	Description
<code>String()</code>	Initializes a newly created <code>String</code> object so that it represents an empty character sequence.
<code>String(byte[] bytes)</code>	Constructs a new <code>String</code> by decoding the specified array of bytes using the platform's default charset.
<code>String(byte[] ascii, int hibyte)</code>	Deprecated. This method does not properly convert bytes into characters.
<code>String(byte[] bytes, int length)</code>	
<code>String(byte[] ascii, int hibyte, int offset, int count)</code>	Deprecated. This method does not properly convert bytes into characters.
<code>String(byte[] bytes, int offset, int length, String charsetName)</code>	Constructs a new <code>String</code> by decoding the specified subarray of bytes using the specified charset.
<code>String(byte[] bytes, int offset, int length, Charset charset)</code>	Constructs a new <code>String</code> by decoding the specified subarray of bytes using the specified charset.

We do not recommend the use of methods / classes / ... that are "deprecated".

Methods of the class String



Return type

Name of the method
with parameters

Method Summary				
All Methods	Static Methods	Instance Methods	Concrete Methods	Deprecated Methods
Modifier and Type	Method	Description		
char	charAt(int index)	Returns the char value at the specified index.		
IntStream	chars()	Returns a stream of int zero-extending the char values from this sequence.		
int	codePointAt(int index)	Returns the character (Unicode code point) at the specified index.		

- Cfr.
public char charAt(int index) {...}

Interface versus implementation



The documentation contains:

- name of the class
- general description of purpose of class
- list of constructors and methods
- return type and parameters for each constructor/method
- description of purpose of each constructor/method

Class interface

The documentation **does not** contain the source code of the class

Implementation of the class

```
public char charAt(int index) {...}
```

A red arrow pointing from the top right towards the closing curly brace of the method signature.

Constructors of class String



- One of the constructors of the class String accepts a parameter of the type String
- For example:
 - String name = new String ("Albert Einstein");
 - The variable *name* now refers to a String object with the value "Albert Einstein".
- Because we use String objects often, we can use in Java the following abbreviated notation: (more memory efficient)
 - String name = "Albert Einstein";

```
String name = new String( original: "Albert Einstein");
```

'new String' is redundant

Replace with argument Alt+Shift+Enter More actions... Alt+Enter

String pool in Java



- Strings are a very common 'data type' in Java and are handled in a special way.
- All String literals e.g. "Albert Einstein" are stored in a String pool
 - The String pool is a store of Strings in a special memory location
 - If you create:
 - String name = "Albert Einstein"
 - Java will check if "Albert Einstein" is already in the String pool. If not, it will add this String to the pool
- If you write: (NOT recommended)
- String name = **new String**("Albert Einstein")
 - An extra String Object is made
 - Even though the String literal "Albert Einstein" already is in the String pool
 - This extra new Object is not needed to create String name (= redundant)

Methods of class String



Click on the link =>
detailed explanation of the method

Brief explanation of the method

String	substring(int beginIndex)	Returns a string that is a substring of this string.
String	substring(int beginIndex, int endIndex)	Returns a string that is a substring of this string.
char[]	toCharArray()	Converts this string to a new character array.
String	toLowerCase()	Converts all of the characters in this <code>String</code> to lower case using the rules of the default locale.
String	toLowerCase(Locale locale)	Converts all of the characters in this <code>String</code> to lower case using the rules of the given <code>Locale</code> .
String	toString()	This object (which is already a string!)
String	toUpperCase()	Converts all of the characters in this <code>String</code> to upper case using the rules of the default locale.
String	toUpperCase(Locale locale)	Converts all of the characters in this <code>String</code> to upper case using the rules of the given <code>Locale</code> .
String	trim()	Returns a string whose value is this string, with all leading and trailing space removed, where space is defined as any character whose codepoint is less than or equal to 'U+0020' (the space character).

Wrapper classes



Purpose of wrapper classes:

- Variables of primitive types can only contain values. They have no associated methods.
- As the name suggests, wrapper classes are **objects encapsulating primitive Java types**.
 - Variables of the 'Wrap class' can hold values, but are also instances of this Wrapper class (with all its methods).
 - Each primitive Java type has a corresponding wrapper:

Primitive type	Wrap class
boolean	Boolean
byte	Byte
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

Automatic boxing and unboxing



- Primitive types can be boxed and unboxed automatically in Java

```
Integer i = 10; // Boxing  
int j = i; // Unboxing
```

```
System.out.println("Value of i: " + i);  
System.out.println("Value of j: " + j);
```

```
Character one = 'a'; // Boxing  
char ch = one; // Unboxing
```

```
System.out.println("Value of one: " + one);  
System.out.println("Value of ch: " + ch);
```

```
Value of i: 10  
Value of j: 10  
Value of one: a  
Value of ch: a
```

Use of wrapper classes



Wrapper classes contain helpful **static methods**:

- `int number = Integer.parseInt(text)`
- `double real = Double.parseDouble(text)`
- `char letter = 'x';`
 if (`Character.isDigit(letter)`)....
 Analogue isLetter(), isWhitespace(),...
- `String input = "true";`
 `boolean choice = Boolean.valueOf(input);`

A **static method** can be **called directly from the Class** (not from an object)

Static method



- `Integer.parseInt(text)`
- `Double.parseDouble(text)`

These methods are examples of class methods or ***static methods***

- You call them up with `Classname.methodname`

Unlike ***instance methods***

- You call them up with `objectname.methodname` (as we did so far)

Method Summary

All Methods	Static Methods	Instance Methods
Modifier	Method	

Static method <-> Instance method



- In Java API you recognise a **class method** by the word **static**.

For example in API of the class Integer:

```
static int parseInt(String s)
```

Parses the string argument as a signed decimal integer.

```
int number = Integer.parseInt ("58");
```

- An **instance method** (without static in the API) is an "ordinary method" that you call using an object / instance e.g. in API of class String:

```
String      substring(int beginIndex, int endIndex)  Returns a string that is a substring of this string.
```

```
String text = "example";  
String result = text.substring(4,7);
```


Exercises on the use of methods



- Requested: make correct use of the following instance or static methods from the Java API of the **class String**:

`boolean`

`endsWith(String suffix)`

Tests if this string ends with the specified suffix.

`static String`

`valueOf(double d)`

Returns the string representation of the `double` argument.

`static String valueOf(boolean b)`

Returns the string representation of the `boolean` argument.

`static String valueOf(char c)`

Returns the string representation of the `char` argument.

`static String valueOf(double d)`

Returns the string representation of the `double` argument.

`static String valueOf(float f)`

Returns the string representation of the `float` argument.

Class LocalDate (in package java.time)



Field Summary

Fields

Modifier and Type	Field	Description
static <code>LocalDate</code>	<code>EPOCH</code>	The epoch year <code>LocalDate</code> , '1970-01-01'.
static <code>LocalDate</code>	<code>MAX</code>	The maximum supported <code>LocalDate</code> , '+999999999-12-31'.
static <code>LocalDate</code>	<code>MIN</code>	The minimum supported <code>LocalDate</code> , '-999999999-01-01'.

Method Summary

All Methods

Static Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
<code>Temporal</code>	<code>adjustInto(Temporal temporal)</code>	Adjusts the specified temporal object to have the same date as this object.
<code>LocalDateTime</code>	<code>atStartOfDay()</code>	Combines this date with the time of midnight to create a <code>LocalDateTime</code> at the start of this date.
<code>ZonedDateTime</code>	<code>atStartOfDay(ZoneId zone)</code>	Returns a zoned date-time from this date at the earliest valid time according to the rules in the time-zone.
<code>LocalDateTime</code>	<code>atTime(int hour, int minute)</code>	Combines this date with a time to create a <code>LocalDateTime</code> .

Class LocalDate



Supports working with dates.

```
import java.time.LocalDate;
```

```
import java.time.Month;
```

Current date:

```
static LocalDate
```

```
now()
```

Obtains the current date from the system clock in the default time-zone.

```
LocalDate today = LocalDate.now();
```

```
System.out.println ("Today is "+ today);
```

```
Today is 2021-01-13
```

Variable with date 15 August 2021:

```
LocalDate d15aug2021 = LocalDate.of(2021, 8, 15);
```

```
LocalDate d15aug2021 = LocalDate.of(2021, Month.AUGUST, 15);
```

```
System.out.println ("We meet at " + d15aug2021);
```

```
We meet at 2021-08-15
```

Class LocalDate



```
System.out.print("Year = "+ today.getYear());
System.out.print(", Month = "+ today.getMonthValue());
System.out.println(", Day = "+ today.getDayOfMonth());
```

Year = 2021, Month = 1, Day = 13

int	getDayOfMonth() Gets the day-of-month field.
DayOfWeek	getDayOfWeek() Gets the day-of-week field, which is an enum DayOfWeek.
int	getDayOfYear() Gets the day-of-year field.
Month	getMonth() Gets the month-of-year field using the Month enum.
int	getMonthValue() Gets the month-of-year field from 1 to 12.
int	getYear() Gets the year field.

From String to LocalDate



```
import java.time.format.DateTimeFormatter;  
import java.time.LocalDate;
```

Scroll down in the api-doc of the class DateTimeFormatter to get to know the meaning and possibilities of the patterns.

```
DateTimeFormatter dtf = DateTimeFormatter.ofPattern("d/M/yyyy");  
LocalDate myDate = LocalDate.parse("1/3/2020", dtf);  
System.out.println(myDate);
```

2020-03-01

For information about the DateTimeFormatter class:
look in the Java API (select the java.time.format package at the top left)

From LocalDate to String



```
LocalDate myDate = LocalDate.parse("2026-03-01");
```

If you would like to print this date in a format of your choice:

```
DateTimeFormatter dtf = DateTimeFormatter.ofPattern ("dd/MM/yyyy");  
System.out.println (myDate.format(dtf));
```

01/03/2026

6

Also, **dtf** has also a method called format().

```
DateTimeFormatter dtf = DateTimeFormatter.ofPattern("d/M/yyyy");  
LocalDate today = LocalDate.now();  
System.out.println(dtf.format(today));
```

17/2/2026

Pattern: "d-M-yyyy"



What does the pattern "d-M-yyyy" mean?

- d – day of the month without a leading zero
- Example: 1, 2, ... 31
- If you used dd, you would get 01, 02, ... 31 with leading zero to enforce 2 digits
- M – month number without a leading zero
- Example: 1, 2, ... 12
- MM would give 01, 02, ... 12
- yyyy – 4-digit year
- Example: 2026
- The hyphens (-) or slashes (/) are literal separators.

1. Using d vs dd
 - d → no leading zero
 - dd → always two digits
2. M (months) vs m (minutes)
 - M → month
 - m → minute and s → seconds

Always use M or MM or MMMM for dates

3. `DateTimeFormatter dtf = DateTimeFormatter.ofPattern("d MMMM yyyy",
Locale.forLanguageTag("nl-BE"));`

`// "1 maart 2026"`

3. ISO 8601 format is safer and better for sorting files:
 - `DateTimeFormatter.ISO_LOCAL_DATE; // "2026-03-01"`